

React Hooks

*Guide approfondi — useState, useEffect, useContext,
useMemo, useCallback, useRef*

Préparé pour un profil Symfony / JS-jQuery / React (Preact)

1. `useState` — l'état local

1.1 Le mécanisme de base

```
import { useState } from 'react';

function Compteur() {
  const [valeur, setValeur] = useState(0);

  return (
    <div>
      <p>Valeur : {valeur}</p>
      <button onClick={() => setValeur(valeur + 1)}>Incrémenter</button>
    </div>
  );
}
```

`useState(0)` retourne un tableau de deux éléments : la valeur actuelle (`valeur`), et une fonction pour la modifier (`setValeur`). Le `0` est la **valeur initiale**, utilisée uniquement au premier rendu.

1.2 Ce qu'il faut vraiment comprendre : le rendu n'est pas immédiat

```
function Exemple() {
  const [compteur, setCompteur] = useState(0);

  const handleClick = () => {
    setCompteur(compteur + 1);
    console.log(compteur); // affiche l'ANCIENNE valeur, pas la nouvelle !
  };

  return <button onClick={handleClick}>{compteur}</button>;
}
```

`setCompteur` ne modifie pas la variable `compteur` sur place — elle **programme un nouveau rendu** du composant, avec une nouvelle valeur. Le `console.log` juste après s'exécute encore dans l'ancien rendu, donc il voit l'ancienne valeur. C'est la source n°1 de confusion chez les développeurs venant de jQuery, où une modification est toujours immédiate et synchrone.

1.3 Piège classique : plusieurs `setState` d'affilée

```
// Ce code N'INCRÉMENTE PAS DE 3, contrairement à l'intuition
const handleClick = () => {
  setCompteur(compteur + 1); // compteur vaut 0 ici (capturé au moment du rendu)
  setCompteur(compteur + 1); // compteur vaut TOUJOURS 0 ici (même closure)
  setCompteur(compteur + 1); // idem
};
// Résultat : compteur passe de 0 à 1, pas à 3 !
```

Chaque appel utilise la valeur de `compteur` **telle qu'elle était au moment où la fonction `handleClick` a été créée** (closure), pas une valeur "à jour" entre chaque ligne. Les trois appels voient tous `compteur = 0`.

La solution : la forme fonctionnelle de `setState`, qui reçoit la valeur la plus récente garantie :

```
const handleClick = () => {
  setCompteur(c => c + 1); // c = 0, résultat 1
  setCompteur(c => c + 1); // c = 1 (la mise à jour précédente est prise en compte), résultat 2
  setCompteur(c => c + 1); // c = 2, résultat 3
};
// Résultat : compteur passe bien de 0 à 3
```

Règle pratique à retenir : si ta nouvelle valeur dépend de l'ancienne, utilise toujours la forme fonctionnelle `setValeur(v => ...)`, jamais `setValeur(valeur + ...)`.

1.4 État avec objets et tableaux : l'immutabilité

React détecte les changements par **comparaison de référence**, pas par inspection profonde du contenu. Modifier un objet/tableau "sur place" ne déclenche **aucun** nouveau rendu :

```
const [utilisateur, setUtilisateur] = useState({ nom: 'Dupont', age: 30 });

// FAUX – ne déclenche pas de re-rendu, et c'est un bug difficile à détecter
const corrigerAge = () => {
  utilisateur.age = 31;
  setUtilisateur(utilisateur); // même référence d'objet, React ne voit aucun changement
};

// CORRECT – nouvelle référence d'objet, créée avec le spread operator
const corrigerAge = () => {
  setUtilisateur({ ...utilisateur, age: 31 });
};
```

Même logique pour les tableaux :

```
const [articles, setArticles] = useState([]);

// FAUX
const ajouterArticle = (nouvel) => {
  articles.push(nouvel); // mutation directe, même référence
  setArticles(articles);
};

// CORRECT
const ajouterArticle = (nouvel) => {
  setArticles([...articles, nouvel]);
};

// CORRECT pour supprimer un élément (par exemple par id)
const supprimerArticle = (id) => {
  setArticles(articles.filter(a => a.id !== id));
};

// CORRECT pour modifier un élément précis dans un tableau d'objets
const modifierPrix = (id, nouveauPrix) => {
  setArticles(articles.map(a =>
    a.id === id ? { ...a, prix: nouveauPrix } : a
  ));
};
```

1.5 Initialisation coûteuse : la forme "lazy"

Si calculer la valeur initiale est coûteux, passer une **fonction** plutôt qu'une valeur directe — elle ne s'exécutera qu'une seule fois (au tout premier rendu), pas à chaque rendu :

```
// Mauvais : recalculeViaLecture() s'exécute À CHAQUE rendu du composant,
// même si seul le résultat du premier appel est utilisé
const [donnees, setDonnees] = useState(recalculeViaLecture());

// Bon : la fonction ne s'exécute qu'au premier rendu
const [donnees, setDonnees] = useState(() => recalculeViaLecture());
```

1.6 Exemple complet : formulaire contrôlé

```
function FormulaireClient() {
  const [client, setClient] = useState({ nom: '', email: '', telephone: '' });

  const handleChange = (champ) => (e) => {
    setClient({ ...client, [champ]: e.target.value });
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    console.log('Envoi :', client);
  };

  return (
```

```
<form onSubmit={handleSubmit}>
  <input value={client.nom} onChange={handleChange('nom')} placeholder="Nom" />
  <input value={client.email} onChange={handleChange('email')} placeholder="Email" />
  <input value={client.telephone} onChange={handleChange('telephone')} placeholder="Téléphone" />
  <button type="submit">Envoyer</button>
</form>
);
}
```

[champ]: `e.target.value` utilise une **clé d'objet calculée dynamiquement** (computed property name du JS moderne) — `handleChange('nom')` retourne une fonction qui, à l'appel, met à jour spécifiquement la clé `nom` de l'objet `client`. Un seul handler générique pour tous les champs du formulaire.

2. `useEffect` — les effets de bord

2.1 Le mécanisme de base

```
import { useEffect, useState } from 'react';

function Horloge() {
  const [heure, setHeure] = useState(new Date());

  useEffect(() => {
    const intervalle = setInterval(() => setHeure(new Date()), 1000);
    return () => clearInterval(intervalle); // nettoyage
  }, []); // tableau de dépendances vide

  return <p>{heure.toLocaleTimeString()}</p>;
}
```

`useEffect` prend une fonction (l'effet) qui s'exécute **après** le rendu (pas pendant), et peut retourner une fonction de **nettoyage**, exécutée avant le prochain effet ou au démontage du composant.

2.2 Le tableau de dépendances : les trois cas

```
// CAS 1 : pas de tableau du tout
useEffect(() => {
  console.log('Je m\'exécute à CHAQUE rendu, sans exception');
});

// CAS 2 : tableau vide []
useEffect(() => {
  console.log('Je m\'exécute UNE SEULE FOIS, au montage du composant');
}, []);

// CAS 3 : tableau avec dépendances
useEffect(() => {
  console.log('Je m\'exécute au montage, ET chaque fois que `userId` change');
}, [userId]);
```

Comment React décide de ré-exécuter l'effet : à chaque rendu, React compare chaque valeur du tableau de dépendances avec sa valeur au rendu précédent (comparaison `Object.is`, similaire à `===`). Si **au moins une** a changé, l'effet se ré-exécute (après avoir d'abord appelé la fonction de nettoyage du rendu précédent, si elle existe).

2.3 Piège majeur : dépendances manquantes

```
function ProfilUtilisateur({ userId }) {
  const [utilisateur, setUtilisateur] = useState(null);

  useEffect(() => {
    fetch(`/api/utilisateurs/${userId}`)
      .then(res => res.json())
      .then(data => setUtilisateur(data));
  }, []); // ⚠️ userId est utilisé dans l'effet mais absent du tableau !

  // ...
}
```

Ce code a un bug réel : si `userId` change (le composant reste monté, mais reçoit une nouvelle prop), l'effet ne se ré-exécute **pas**, parce que le tableau de dépendances est vide. Le composant affichera les données du **premier** `userId` reçu, indéfiniment.

Correction :

```
useEffect(() => {
  fetch(`/api/utilisateurs/${userId}`)
    .then(res => res.json())
```

```
.then(data => setUtilisateur(data));
}, [userId]); // userId ajouté : l'effet se ré-exécute bien à chaque changement
```

Règle pratique : toute variable (prop, state, ou valeur calculée) **lue à l'intérieur** de l'effet doit figurer dans le tableau de dépendances. Le plugin ESLint `eslint-plugin-react-hooks` (règle `exhaustive-deps`) détecte ça automatiquement — ne jamais désactiver ce warning sans en comprendre la raison exacte.

2.4 Le nettoyage (cleanup) : pourquoi c'est important

```
function ComposantAvecAbonnement({ canalId }) {
  useEffect(() => {
    const abonnement = serviceMessages.sabonner(canalId, (msg) => {
      console.log('Nouveau message :', msg);
    });

    return () => {
      abonnement.annuler(); // ESSENTIEL : sinon fuite mémoire / abonnements multiples
    };
  }, [canalId]);

  // ...
}
```

Sans cette fonction de nettoyage : si `canalId` change, un **nouvel** abonnement est créé à chaque fois, sans jamais fermer les précédents — fuite de ressources, et le composant reçoit des messages de plusieurs canaux à la fois alors qu'il ne devrait écouter que le dernier.

Quand le nettoyage s'exécute, précisément : 1. Juste avant que l'effet ne se ré-exécute (si une dépendance a changé) 2. Au démontage complet du composant

2.5 Cas pratique très fréquent : appel API avec gestion de l'annulation

Un piège réel en production : si le composant est démonté (ou la dépendance change) **avant** que la requête réseau ne soit terminée, le `setState` qui arrive après peut s'exécuter sur un composant qui n'existe plus, causant un avertissement React ("Cannot update state on an unmounted component") et potentiellement un état incohérent.

```
function ListeInterventions({ jour }) {
  const [interventions, setInterventions] = useState([]);

  useEffect(() => {
    let annule = false;

    fetch(`/api/interventions?jour=${jour}`)
      .then(res => res.json())
      .then(data => {
        if (!annule) { // on ne met à jour l'état QUE si l'effet n'a pas été "annulé"
          setInterventions(data);
        }
      });

    return () => {
      annule = true; // marque cet effet comme obsolète avant le prochain
    };
  }, [jour]);

  return (
    <ul>
      {interventions.map(i => <li key={i.id}>{i.libelle}</li>)}
    </ul>
  );
}
```

Ce pattern (variable `annule / ignore` locale à l'effet) est un classique à connaître pour tout composant qui fait un appel réseau dans un `useEffect`.

2.6 `useEffect` n'est pas fait pour tout

Erreur fréquente : utiliser `useEffect` pour calculer une valeur dérivée d'un autre état.

```
// Mauvais : un useEffect inutile, qui ajoute un rendu supplémentaire
const [articles, setArticles] = useState([]);
const [total, setTotal] = useState(0);

useEffect(() => {
  setTotal(articles.reduce((acc, a) => acc + a.prix, 0));
}, [articles]);

// Bon : calcul direct au rendu, pas de useEffect, pas de state supplémentaire
const [articles, setArticles] = useState([]);
const total = articles.reduce((acc, a) => acc + a.prix, 0);
```

`useEffect` doit servir pour de vrais effets de bord (réseau, abonnements, manipulation directe d'API externes) — pas pour synchroniser deux states entre eux, ce qui peut presque toujours être remplacé par un calcul direct (voir aussi `useMemo`, section 4, si ce calcul est coûteux).

3. `useContext` — partager un état sans prop drilling

3.1 Le problème que `useContext` résout

```
// Sans context : "prop drilling" — la prop traverse des composants qui n'en ont pas besoin
function App() {
  const [theme, setTheme] = useState('clair');
  return <Page theme={theme} />;
}
function Page({ theme }) {
  return <Header theme={theme} />; // Page n'utilise pas theme, juste le retransmet
}
function Header({ theme }) {
  return <BoutonConnexion theme={theme} />; // pareil
}
function BoutonConnexion({ theme }) {
  return <button className={theme}>Connexion</button>; // ENFIN utilisé ici
}
```

3.2 Le mécanisme de base

```
import { createContext, useContext, useState } from 'react';

const ThemeContext = createContext();

function App() {
  const [theme, setTheme] = useState('clair');

  return (
    <ThemeContext.Provider value={{ theme, setTheme }}>
      <Page />
    </ThemeContext.Provider>
  );
}

function Page() {
  return <Header />; // n'a plus besoin de connaître `theme`
}

function Header() {
  return <BoutonConnexion />;
}

function BoutonConnexion() {
  const { theme, setTheme } = useContext(ThemeContext); // accès direct, peu importe la profondeur
  return (
    <button
      className={theme}
      onClick={() => setTheme(theme === 'clair' ? 'sombre' : 'clair')}
    >
      Connexion
    </button>
  );
}
```

`Page` et `Header` n'ont plus besoin de connaître `theme` du tout — seul `BoutonConnexion`, qui en a réellement besoin, l'utilise via `useContext`.

3.3 Pattern recommandé : un hook personnalisé pour chaque contexte

Plutôt que d'appeler `useContext(ThemeContext)` partout (et de devoir importer `ThemeContext` à chaque fois), créer un petit hook dédié :

```
// fichier ThemeContext.js
import { createContext, useContext } from 'react';

export const ThemeContext = createContext();

export function useTheme() {
  const contexte = useContext(ThemeContext);
}
```



```

if (contexte === undefined) {
  throw new Error('useTheme doit être utilisé à l'intérieur d'un ThemeContext.Provider');
}
return contexte;
}

// utilisation ailleurs dans l'app
import { useTheme } from './ThemeContext';

function BoutonConnexion() {
  const { theme, setTheme } = useTheme(); // plus propre, avec garde-fou intégré
  // ...
}

```

Ce garde-fou (`throw new Error` si le contexte est `undefined`) évite un bug silencieux fréquent : utiliser le hook en dehors de son `Provider`, ce qui sinon retournerait simplement `undefined` sans erreur claire, et planterait plus loin de façon confuse.

3.4 Piège de performance : un seul `Provider`, tous les composants qui le consomment se re-rendent

```

// Tous les composants qui font `useContext(AppContext)` se re-rendent
// dès que N'IMPORTE QUELLE partie de `value` change, même celle qu'ils n'utilisent pas
<AppContext.Provider value={{ theme, utilisateur, panier, notifications }}>

```

Si un composant ne lit que `theme` via ce contexte, mais que `panier` change ailleurs dans l'app, ce composant se re-rend quand même inutilement. Pour des contextes avec plusieurs valeurs indépendantes qui changent à des fréquences différentes, mieux vaut **plusieurs contextes séparés** plutôt qu'un seul gros contexte global :

```

<ThemeContext.Provider value={theme}>
  <PanierContext.Provider value={panier}>
    <App />
  </PanierContext.Provider>
</ThemeContext.Provider>

```

3.5 Exemple complet : contexte d'authentification

```

import { createContext, useContext, useState } from 'react';

const AuthContext = createContext();

export function AuthProvider({ children }) {
  const [utilisateur, setUtilisateur] = useState(null);

  const connexion = async (email, motDePasse) => {
    const reponse = await fetch('/api/login', {
      method: 'POST',
      body: JSON.stringify({ email, motDePasse }),
    });
    const data = await reponse.json();
    setUtilisateur(data.utilisateur);
  };

  const deconnexion = () => setUtilisateur(null);

  return (
    <AuthContext.Provider value={{ utilisateur, connexion, deconnexion }}>
      {children}
    </AuthContext.Provider>
  );
}

export function useAuth() {
  return useContext(AuthContext);
}

// Utilisation dans n'importe quel composant de l'app, sans prop drilling :
function BarreNavigation() {
  const { utilisateur, deconnexion } = useAuth();
  return utilisateur
}

```

```
? <button onClick={deconnexion}>Déconnexion ({utilisateur.nom})</button>  
: <p>Non connecté</p>;  
}
```

4. useMemo — mémoriser un résultat de calcul

4.1 Le problème que useMemo résout

À chaque rendu d'un composant, **tout le code de la fonction se ré-exécute**, y compris les calculs coûteux, même si leurs entrées n'ont pas changé :

```
function ListeArticles({ articles, filtre }) {  
  // Ce filtrage s'exécute à CHAQUE rendu du composant,  
  // même si articles et filtre n'ont pas changé (par exemple si un état  
  // complètement différent du composant parent a changé)  
  const articlesFiltres = articles.filter(a => a.nom.includes(filtre));  
  
  return <ul>{articlesFiltres.map(a => <li key={a.id}>{a.nom}</li>)}</ul>;  
}
```

Pour un filtrage simple sur peu d'articles, ce n'est pas un problème réel. Mais pour un calcul réellement coûteux (tri complexe, agrégations sur de gros volumes), ça peut dégrader les performances perceptiblement.

4.2 Le mécanisme de base

```
import { useMemo } from 'react';  
  
function ListeArticles({ articles, filtre }) {  
  const articlesFiltres = useMemo(() => {  
    return articles.filter(a => a.nom.includes(filtre));  
  }, [articles, filtre]); // recalcule SEULEMENT si articles ou filtre changent  
  
  return <ul>{articlesFiltres.map(a => <li key={a.id}>{a.nom}</li>)}</ul>;  
}
```

Si le composant se re-rend pour une raison qui n'a rien à voir avec `articles` ou `filtre` (un autre state du même composant qui change, par exemple), `useMemo` retourne directement le résultat **déjà calculé** au rendu précédent, sans ré-exécuter le filtrage.

4.3 Piège : useMemo n'est pas gratuit

```
// Inutile : additionner deux nombres est quasi instantané,  
// l'overhead de useMemo (comparaison des dépendances, stockage en mémoire)  
// coûte probablement plus cher que le calcul lui-même  
const somme = useMemo(() => a + b, [a, b]);  
  
// Suffisant  
const somme = a + b;
```

Règle pratique : n'utiliser `useMemo` que pour des calculs **mesurablement coûteux** (boucles sur de gros tableaux, calculs récursifs, etc.), ou pour préserver une **référence stable** d'un objet/tableau passé en prop à un composant enfant optimisé (voir aussi 5.3).

4.4 Exemple avec un calcul réellement coûteux

```
function TableauStatistiques({ ventes }) {  
  const statistiques = useMemo(() => {  
    console.log('Calcul des statistiques...'); // pour observer quand ça s'exécute réellement  
    return {  
      total: ventes.reduce((acc, v) => acc + v.montant, 0),  
      moyenne: ventes.reduce((acc, v) => acc + v.montant, 0) / ventes.length,  
      meilleureVente: ventes.reduce((max, v) => v.montant > max.montant ? v : max, ventes[0]),  
    };  
  }, [ventes]);  
  
  return (  
    <div>
```

```
<p>Total : {statistiques.total} €</p>
<p>Moyenne : {statistiques.moyenne.toFixed(2)} €</p>
<p>Meilleure vente : {statistiques.meilleureVente.libelle}</p>
</div>
);
}
```

5. `useCallback` — mémoriser une fonction

5.1 Le problème que `useCallback` résout

À chaque rendu, une fonction définie dans le corps du composant est **recréée** — une nouvelle référence en mémoire, même si son code est identique :

```
function ListeArticles({ articles }) {  
  // Cette fonction est une NOUVELLE fonction à chaque rendu de ListeArticles  
  const gererClic = (id) => {  
    console.log('Article cliqué :', id);  
  };  
  
  return articles.map(a => (  
    <ArticleOptimise key={a.id} article={a} onClick={gererClic} />  
  ));  
}
```

Si `ArticleOptimise` est enveloppé dans `React.memo` (pour éviter de se re-rendre quand ses props n'ont "pas changé"), le fait que `gererClic` soit une **nouvelle référence** à chaque rendu de `ListeArticles` fait que `React.memo` considère que la prop `onClick` a changé — et `ArticleOptimise` se re-rend quand même, ce qui annule l'optimisation recherchée.

5.2 Le mécanisme de base

```
import { useCallback } from 'react';  
  
function ListeArticles({ articles }) {  
  const gererClic = useCallback((id) => {  
    console.log('Article cliqué :', id);  
  }, []); // référence stable, ne change jamais (tant que les dépendances ne changent pas)  
  
  return articles.map(a => (  
    <ArticleOptimise key={a.id} article={a} onClick={gererClic} />  
  ));  
}
```

5.3 `useCallback` n'a de sens qu'avec `React.memo` (ou des dépendances d'effets)

```
const ArticleOptimise = React.memo(function ArticleOptimise({ article, onClick }) {  
  console.log('Rendu de', article.nom); // pour observer les re-rendus  
  return <li onClick={() => onClick(article.id)}>{article.nom}</li>;  
});
```

Sans `React.memo` sur `ArticleOptimise`, utiliser `useCallback` côté parent n'apporte **aucun bénéfice** — le composant enfant se re-rendrait de toute façon à chaque rendu du parent, peu importe la stabilité de la référence de la fonction. C'est l'erreur la plus fréquente : mettre `useCallback` "par réflexe" sans l'associer à `React.memo` côté composant enfant, ou sans que la fonction soit utilisée comme dépendance d'un autre `useEffect`.

5.4 Cas d'usage légitime n°2 : dépendance d'un `useEffect`

```
function Recherche({ terme }) {  
  // Sans useCallback, rechercherApi serait une nouvelle fonction à chaque rendu,  
  // ce qui ferait que l'effet ci-dessous se ré-exécuterait à CHAQUE rendu  
  // (puisque rechercherApi serait "différente" à chaque fois)  
  const rechercherApi = useCallback(async (q) => {  
    const res = await fetch(`/api/recherche?q=${q}`);  
    return res.json();  
  }, []);  
  
  useEffect(() => {  
    rechercherApi(terme).then(resultats => console.log(resultats));  
  }, [terme, rechercherApi]); // rechercherApi est stable grâce à useCallback
```

```
return null;  
}
```

5.5 `useMemo` vs `useCallback` : la différence en une phrase

`useCallback(fn, deps)` est strictement équivalent à `useMemo(() => fn, deps)` — la seule différence est que `useCallback` mémorise directement une **fonction**, alors que `useMemo` mémorise le **résultat** d'une fonction qu'on lui passe.

6. useRef — référence mutable qui ne déclenche pas de rendu

6.1 Le mécanisme de base

```
import { useRef } from 'react';

function ChampAvecFocus() {
  const inputRef = useRef(null);

  const focaliser = () => {
    inputRef.current.focus(); // accès direct à l'élément DOM réel
  };

  return (
    <div>
      <input ref={inputRef} type="text" />
      <button onClick={focaliser}>Focaliser le champ</button>
    </div>
  );
}
```

`useRef(null)` crée un objet `{ current: null }` qui **persiste** entre les rendus (contrairement à une variable normale, recrée à chaque rendu). En l'attachant via `ref={inputRef}` à un élément JSX, React remplit automatiquement `inputRef.current` avec le **vrai nœud DOM** correspondant.

6.2 La différence fondamentale avec `useState`

```
function Exemple() {
  const compteurState = useState(0); // déclenche un RENDU quand on le modifie
  const compteurRef = useRef(0);    // ne déclenche AUCUN rendu quand on le modifie

  const incrementerRef = () => {
    compteurRef.current += 1;
    console.log(compteurRef.current); // changement immédiatement visible ici...
    // ...mais l'affichage à l'écran (si on affichait {compteurRef.current} en JSX)
    // ne se mettrait PAS à jour visuellement, car aucun rendu n'est déclenché
  };
}
```

Règle de décision : utiliser `useState` si la valeur doit être **affichée** dans l'UI (donc nécessite un rendu pour refléter le changement). Utiliser `useRef` pour une valeur qui doit **persister** entre les rendus mais qui n'a pas besoin de déclencher un ré-affichage (un timer ID, un compteur de rendus pour du debug, une valeur "précédente" à comparer, l'accès à un élément DOM).

6.3 Cas d'usage n°2 : stocker une valeur sans redéclencher de rendu

```
function MinuteurAffichage({ valeur }) {
  const valeurPrecedente = useRef();

  useEffect(() => {
    valeurPrecedente.current = valeur; // mis à jour après chaque rendu
  }, [valeur]);

  return (
    <p>
      Valeur actuelle : {valeur}, précédente : {valeurPrecedente.current}
    </p>
  );
}
```

6.4 Cas d'usage n°3 : éviter une exécution en double dans `useEffect` (pattern courant avec React 18 StrictMode)

```
function ComposantAvecInitUnique() {
  const dejaInitialise = useRef(false);
```

```

useEffect(() => {
  if (dejaInitialise.current) return; // évite une double exécution
  dejaInitialise.current = true;

  console.log('Initialisation unique, même en StrictMode');
}, []);

return null;
}

```

(Contexte : en mode développement avec `StrictMode`, React 18 exécute volontairement certains effets deux fois pour aider à détecter les effets non idempotents — ce pattern avec `useRef` est une façon courante de neutraliser ce comportement pour une init qui doit vraiment être unique, comme l'ouverture d'une connexion.)

6.5 Exemple complet : chronomètre avec start/stop

```

function Chronometre() {
  const [tempsEcoule, setTempsEcoule] = useState(0);
  const [enCours, setEnCours] = useState(false);
  const intervalleRef = useRef(null);

  const demarrer = () => {
    setEnCours(true);
    intervalleRef.current = setInterval(() => {
      setTempsEcoule(t => t + 1);
    }, 1000);
  };

  const arreter = () => {
    setEnCours(false);
    clearInterval(intervalleRef.current); // accès à l'ID stocké, sans déclencher de rendu
  };

  useEffect(() => {
    return () => clearInterval(intervalleRef.current); // nettoyage au démontage
  }, []);

  return (
    <div>
      <p>{tempsEcoule} secondes</p>
      <button onClick={enCours ? arreter : demarrer}>
        {enCours ? 'Arrêter' : 'Démarrer'}
      </button>
    </div>
  );
}

```

`intervalleRef` stocke l'ID retourné par `setInterval` — une valeur purement technique qui n'a aucune raison de déclencher un rendu quand elle change (contrairement à `tempsEcoule`, qui lui doit bien être un `useState` puisqu'il est affiché).

Synthèse : quel hook pour quel besoin

Besoin	Hook
Une valeur qui doit s'afficher et changer dans l'UI	<code>useState</code>
Réagir à un changement, faire un appel réseau, s'abonner à quelque chose	<code>useEffect</code>
Partager une valeur sans la faire transiter par toutes les props intermédiaires	<code>useContext</code>
Éviter de recalculer une valeur coûteuse à chaque rendu	<code>useMemo</code>
Éviter de recréer une fonction à chaque rendu (pour <code>React.memo</code> ou dépendance de <code>useEffect</code>)	<code>useCallback</code>
Garder une valeur entre les rendus sans déclencher de rendu, ou accéder à un élément DOM	<code>useRef</code>